Facial Landmark Detection

1. Content Description

This course implements the function of acquiring color images and using the MediaPipe framework to detect faces. This section requires entering commands in the terminal. The appropriate terminal will be selected based on the motherboard type. This section uses a Raspberry Pi 5 as an example.

For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this section in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**.

For Orin and RDK motherboards, simply open the terminal and enter the commands mentioned in this section.

2. Program Startup

For the Raspberry Pi 5 and jetson nano controller, you must first enter the Docker container. For the Orin and RDK controller, this is not necessary.

Entering the Docker container (see the Docker course for steps: 4. Docker Startup Script).

All commands below must be executed within the same Docker container.** (See the Docker course for steps: 3. Docker Commit and Multi-Terminal Entry).

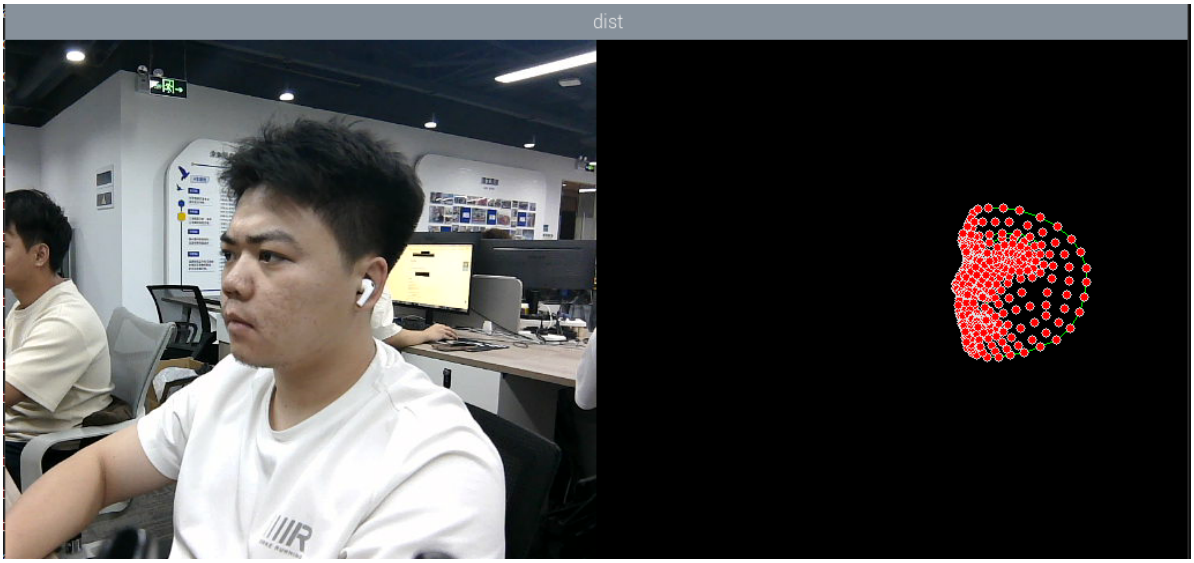
First, in the terminal, enter the following command to start the camera.

```
#usb camera
ros2 launch usb_cam camera.launch.py
#nuwa camera
ros2 launch ascamera hp60c.launch.py
```

After successfully starting the camera, open another terminal and enter the following command to start the face detection program.

```
ros2 run yahboomcar_mediapipe 04_FaceMesh
```

After running the program, the image below shows the detected facial points on the right side of the image.



3、核心代码解析

程序代码路径：

- 树莓派5和Jetson-Nano主板

  程序代码在运行的docker中。docker中的路径为`/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/04_FaceMesh.py`

- Orin主板

  程序代码路径为`/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/04_FaceMesh.py`

- RDK主板

  程序代码路径为`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/04_FaceMesh.py`

导入用到的库文件，

```
import time
import rclpy
from rclpy.node import Node
import cv2 as cv
import numpy as np
import mediapipe as mp
from geometry_msgs.msg import Point
from yahboomcar_msgs.msg import PointArray
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import os
```

Initialize data and define publishers and subscribers,

```
def __init__(self, staticMode=False, maxFaces=2, minDetectionCon=0.5,
minTrackingCon=0.5):
    super().__init__('face_mesh_detector')
    self.mpDraw = mp.solutions.drawing_utils
```

```
    #Use the class in the mediapipe library to define a face object
    self.mpFaceMesh = mp.solutions.face_mesh
    self.faceMesh = self.mpFaceMesh.FaceMesh(
    static_image_mode=staticMode,
    max_num_faces=maxFaces,
    min_detection_confidence=minDetectionCon,
    min_tracking_confidence=minTrackingCon )
    #Define the properties of the joint connection line, which will be used in
the subsequent joint point connection function
    self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
thickness=-1, circle_radius=3)
    self.drawSpec = self.mpDraw.DrawingSpec(color=(0, 255, 0), thickness=1,
circle_radius=1)
    self.bridge = CvBridge()
    #Define subscribers for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type ==
'nuwa' else '/usb_cam/image_raw'
    self.subscription = self.create_subscription(
    Image,
    topic_name,
    self.image_callback,
    10)
```

Color image callback function,

```
def get_RGBImageCallBack(self,msg):
    #Use CvBridge to convert color image message data into image data
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    #Pass the obtained image into the defined pubFaceMeshPoint function,
draw=False means not to draw joint points on the original color image
    frame, img = self.pubFaceMeshPoint(frame, draw=True)
    #Merge two images
    dist = self.frame_combine(frame, img)
    key = cv2.waitKey(1)
    cv.imshow('dist', dist)
```

pubFaceMeshPoint function,

```
def pubFaceMeshPoint(self, frame, draw=True):
    #Create a new image based on the incoming image size. The image data type is
uint8
    img = np.zeros(frame.shape, np.uint8)
    #Convert the color space of the incoming image from BGR to RGB to facilitate
subsequent image processing
    imgRGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library for image processing.
During init, the self.faceMesh object is created and initialized.
    self.results = self.faceMesh.process(imgRGB)
    #Judge whether self.results.multi_face_landmarks exists, that is, whether the
face is recognized
    if self.results.multi_face_landmarks:
        for i in range(len(self.results.multi_face_landmarks)):
            if draw: self.mpDraw.draw_landmarks(frame,
self.results.multi_face_landmarks[i], self.mpFaceMesh.FACEMESH_CONTOURS,
self.lmDrawSpec, self.drawSpec)
```

```
            #Connect each joint point on the blank image created previously
            self.mpDraw.draw_landmarks(img,
self.results.multi_face_landmarks[i], self.mpFaceMesh.FACEMESH_CONTOURS,
self.lmDrawSpec, self.drawSpec)
    return frame, img
```

The frame_combine image merging function was mentioned in the first lesson of this chapter. Please refer to [07.Mediapipe Visual Course] - [1. Hand Detection] for an analysis of this function.